Representation-Induced State Divergence in Deterministic CDCL SAT Solving

Tracing the Earliest Observable Differences Between Semantically Equivalent Executions

Syed Faizyab Mujtaba

Independent Research

Abstract

Variable renaming is logically irrelevant to a Boolean satisfiability problem, yet it can substantially alter the behavior of a deterministic CDCL SAT solver. This study investigates how such a representational perturbation propagates into different computational search trajectories. Five SAT instances from the ROCSAT benchmark corpus were each subjected to 100 single-variable transpositions, with every resulting instance solved independently using the same MiniSat 2.2.0 configuration. Solver effort was measured using decisions, conflicts, propagations, and elapsed execution time, while selected executions were additionally traced at the level of branching decisions, variable activities, priority-structure positions, and conflict states.

The experiments show substantial variation across semantically equivalent representations. On `bmc-ibm-13.cnf`, observed decision counts ranged from 2,845 to 62,320 compared with 49,726 for the baseline. Other benchmarks exhibited distinct sensitivity patterns, including a pronounced high-effort tail for `3bitadd_31.cnf` and comparatively concentrated behavior for `aim-100-1_6-no-1.cnf`. Mechanistic analysis identified an early divergence in which two equally active variables occupied reversed positions in the branching priority structure, producing different branching decisions and immediately different conflict-learning behavior. Other high-effort permutations remained aligned with the baseline under the recorded decision and conflict traces for substantial portions of the search, indicating that the same mechanism cannot explain every divergence.

These results demonstrate that semantic equivalence does not imply computational equivalence for a stateful deterministic CDCL solver. A small representational perturbation can alter concrete internal state and, through subsequent heuristic and search transitions, develop into a substantially different computational trajectory. The evidence therefore supports path-dependent, instance-dependent representation sensitivity rather than a single universal mechanism.

1. Introduction

A variable name has no meaning to a Boolean formula, but it can acquire meaning to the program that solves the formula.

Under a bijective renaming of its variables, a CNF formula preserves its satisfiability and its logical relationships. From the perspective of the problem being solved, the two representations are interchangeable. A concrete CDCL solver, however, does not reason over an abstract equivalence class of formulas. It maintains mutable state over a particular representation: variables accumulate heuristic activity, branching candidates are ordered, assignments trigger propagation, conflicts generate learned clauses, and the consequences of these operations determine subsequent decisions. The resulting computation is therefore sensitive not only to the constraints themselves, but to the state from which the solver encounters them.

The practical consequence is well established: changing the representation of a CNF instance can change the amount of work required by a SAT solver. Biere and Heule's study of CNF scrambling, for example, found substantial effects of scrambling on the performance of individual solvers and introduced controlled scrambling precisely to study the robustness of SAT evaluation. [3] That result raises a different

question from the one addressed here. Performance variation tells us that the representation matters; it does not tell us how the representation becomes computationally consequential while solving.

A solver run is not a scalar measurement but a sequence of state transitions. A branching decision changes the assignment state; propagation exposes consequences; a conflict triggers analysis and clause learning; the learned clause and updated heuristic state then influence later branching. If two executions of a deterministic solver eventually require different numbers of decisions, conflicts, or propagations, some difference has propagated through this state-transition process. The interesting problem is to locate and characterize that propagation.

This distinction is easy to miss if representation sensitivity is studied only through final runtime. Suppose two equivalent instances produce different first decisions. The resulting trajectories may immediately separate, making the initial ordering difference a plausible source of the divergence. But the converse is more difficult: two executions may make the same observed decisions for a substantial period and nevertheless accumulate enough hidden state difference to produce different computational outcomes later. In that case, the first visible divergence is not necessarily the first causal divergence.

This study uses variable renaming as a controlled perturbation of that process. A fixed CNF instance is transformed by swapping variable identities while leaving its clause structure unchanged. The transformed and original instances therefore differ in representation but not in the Boolean problem they encode. Both are executed using the same deterministic MiniSat configuration. Rather than recording only whether one representation is faster, the experiment follows the executions through solver-level measures of search effort and, for selected cases, through branching decisions and internal heuristic state.

Prior work establishes that representational changes can affect solver performance [3], while work on CDCL heuristics demonstrates that variable-selection behavior can have a substantial effect on the resulting search process [4]. The central question addressed here is not whether renaming changes performance, but how a representation-level perturbation propagates through a stateful heuristic algorithm.

This produces a more specific object of study: the search trajectory of a deterministic CDCL solver under semantically equivalent representations. In particular, we ask whether equivalent executions diverge through an identifiable early event, whether divergence can remain hidden at the level of observable decisions, and whether apparently different performance outcomes can be traced to a common internal mechanism.

The experiments show that the answer is not reducible to a single immediate branching effect. Across 100 single-variable permutations of a fixed benchmark, solver effort spans a wide range even though the underlying Boolean problem is preserved. One permutation requires only 2,845 decisions compared with 49,726 for the original representation, while another requires 62,320. More importantly, trajectory analysis exposes qualitatively different forms of divergence. In one case, an ordering difference between equally active branching candidates is followed by an altered decision and a different conflict trajectory. In another, the recorded decision sequence remains aligned with the original execution through a substantial portion of the search even though the two runs eventually differ substantially in total effort.

These observations motivate a view of representation sensitivity as computational path dependence rather than a simple performance anomaly. The variable permutation does not alter the logical difficulty of the Boolean problem in the semantic sense. It perturbs the concrete state of a heuristic algorithm, and the subsequent sequence of state transitions can amplify that perturbation into a different computational history. The purpose of this work is to characterize that amplification and, where possible, identify the internal events through which it occurs.

2. Background

2.1. Boolean Satisfiability

Boolean satisfiability, or SAT, is the problem of determining whether a Boolean formula admits an assignment of truth values under which the entire formula evaluates to true. A SAT instance is therefore defined by a collection of Boolean variables together with the logical constraints imposed on those variables. The identifiers used to refer to the variables are not part of those constraints in themselves; they are labels used to represent the underlying Boolean relationships.

The experiments in this work use formulas in conjunctive normal form (CNF). A CNF formula is an AND of clauses, while each clause is an OR of literals. A literal is either a variable or its negation. For example,

$$F = (x_1 \vee \neg x_3 \vee x_7) \wedge (\neg x_1 \vee x_4) \wedge (x_2 \vee \neg x_4 \vee x_6).$$

An assignment satisfies F only when every clause contains at least one true literal. The variables themselves have no intrinsic ordering or semantic identity. The role of x_1, for example, is determined entirely by where it occurs and whether those occurrences are negated, not by the fact that the variable is called x_1 rather than x_{17}.

This makes variable renaming a particularly simple way of changing the representation of a SAT instance without changing the problem it expresses. Let the variables of F be

$$V = \{x_1, x_2, \ldots, x_n\}.$$

A variable permutation rearranges these variable identities so that each variable is replaced by another variable in V, with every variable used exactly once.

$$\pi : V \to V$$

defines a renaming of the variables. Applying π to a formula means replacing every occurrence of a variable x_i with $\pi(x_i)$, including occurrences inside negated literals. The structure of the formula is otherwise unchanged. For example, swapping x_2 and x_7 would replace every occurrence of x_2 with x_7 and every occurrence of x_7 with x_2, without changing the number of clauses, the number of literals, or the pattern of connections between variables and clauses.

The important point is that this transformation does not create or remove solutions. Suppose an assignment α satisfies the original formula F. Applying the same permutation to the variable assignments produces a corresponding assignment α_π for the renamed formula. Because every variable has exactly one image under a bijection, and every variable has exactly one preimage, this correspondence works in both directions. Consequently,

$$F \text{ is satisfiable} \iff F_\pi \text{ is satisfiable.}$$

More strongly, the two instances have the same logical structure up to a change of variable names. A variable that participates in a particular set of clauses before the transformation participates in the corresponding set of renamed clauses afterward. What changes is therefore not the Boolean problem encoded by the formula, but the identifiers through which that problem is presented.

This distinction is important for the experiments that follow. A SAT solver does not receive an abstract statement that two variables are the same kind of object under relabeling; it receives a representation of the instance and constructs its internal data structures from that representation. The variable identifiers consequently provide a controlled point at which two logically equivalent instances can be made different before solving begins.

Variable permutation is therefore useful here precisely because it isolates a narrow form of representa-

tional change. The clauses and their logical relationships are preserved, while the identities attached to the variables are rearranged. Any subsequent difference in the behavior of a fixed solver can then be examined as a consequence of how that solver interacts with the representation, rather than as a consequence of having supplied it with a different Boolean problem.

2.2. Conflict-Driven Clause Learning

A CDCL solver searches for a satisfying assignment by maintaining a partial assignment to the variables and extending that assignment through a sequence of decisions and deductions. When the current assignment leads to a contradiction, the solver does not simply discard the entire search path. Instead, it analyzes the conflict, learns a new clause that prevents the same combination of assignments from being reproduced, and continues the search from an earlier decision level.

The search begins with a decision. The solver selects an unassigned variable, assigns it a truth value, and records the assignment at the current decision level. This assignment may immediately force other variables to take particular values. Such forced assignments are obtained through Boolean constraint propagation (BCP): whenever all but one literal in a clause are false, the remaining literal must be true in order for the clause to remain satisfied.

For example, consider the clause

$$(x_1 \vee \neg x_2 \vee x_3).$$

If the current assignment makes x_1 false and x_2 true, then the first two literals are false. The clause can therefore only be satisfied if x_3 is true, so propagation assigns x_3 accordingly. A single decision can consequently trigger a chain of additional assignments without any further branching decisions.

Propagation can eventually produce a conflict when a clause becomes unsatisfied under the current assignment. At that point, the solver has reached a state that cannot lead to a solution under the current sequence of decisions. CDCL responds by analyzing the assignments and implications that led to the conflict. The result is a learned clause containing a combination of literals that cannot simultaneously occur in any valid solution under the same constraints.

The solver then backtracks to an earlier decision level and continues the search with the learned clause added to its clause database. The learned clause is not merely a record of the conflict that has already occurred; it becomes an additional constraint that can participate in propagation and future conflict analysis. Consequently, an event that occurs at one point in the search can influence the solver much later in the same execution.

The sequence is not fixed in advance. Which variable is selected for a decision, which assignments are produced by propagation, which conflict is encountered, and which clause is learned all depend on the state accumulated during the preceding search.

This statefulness is important for the experiment in this work. Two semantically equivalent formulas may contain exactly the same logical constraints while causing the solver to encounter those constraints through different concrete variable identities and internal data structures. Once a difference affects a decision or propagation event, the resulting learned clauses and assignments can alter later states of the search. A small difference near the beginning of an execution can therefore influence events far beyond the point at which it first occurred.

Modern CDCL solvers combine Boolean constraint propagation, conflict analysis, clause learning, and heuristic variable selection to navigate the search space. The engineering of these components was famously demonstrated by Chaff, which emphasized efficient propagation and a low-overhead decision strategy [1]. MiniSat subsequently provided a compact and extensible implementation of conflict-driven learning that became widely used as a reference SAT solver [2].

2.3. Heuristic State and Search Trajectories

The sequence of decisions made by a CDCL solver is influenced by more than the clauses in the input formula. The solver maintains heuristic information that changes throughout the search and uses that information to determine which unassigned variable should be selected next. In MiniSat, this includes a numerical activity associated with each variable and a priority structure used to select a high-priority branching candidate.

Variable activity is updated as the solver encounters conflicts. Variables that participate in conflict analysis can receive increased activity, causing them to become more likely to be selected in subsequent decisions [4]. Activity therefore reflects information accumulated from the solver's previous search history rather than a fixed property of the original formula.

The branching structure introduces another aspect of the solver state. Even when two variables have identical activity values, their relative positions inside the priority structure can affect which one is selected. The choice made at a decision point therefore depends on the concrete state of the solver at that moment, including both heuristic scores and the structure used to organize candidate variables.

This matters because a branching decision changes the state on which all subsequent operations depend. Different decisions can produce different propagation chains and different conflicts. Those conflicts can lead to different learned clauses and different activity updates, which in turn modify later branching decisions. The solver consequently creates a feedback loop in which its previous behavior influences the conditions under which its future behavior occurs.

For a fixed input representation and deterministic configuration, the solver's behavior is deterministic: once its state is fixed, the subsequent state transitions are fixed as well. This does not mean that two equivalent representations must generate the same trajectory. A variable permutation can alter the concrete representation from which the solver constructs its internal state. If this produces even a small difference in heuristic state or branching order, the difference can propagate through subsequent search steps.

It is useful to distinguish three related but different objects when comparing two solver executions: The first is the *solver state*, which includes the assignments, heuristic information, learned clauses, and other mutable data maintained during search. The second is the *search trajectory*, the sequence of decisions, propagations, conflicts, backtracks, and learned clauses produced as that state evolves. The third is the resulting *computational effort*, measured here through quantities such as the total number of decisions, conflicts, propagations, and elapsed time.

These quantities do not necessarily diverge at the same point. Two executions can have different internal states while continuing to make the same observed decisions, or they can make different decisions and immediately enter different conflict trajectories. Likewise, a difference in an early event does not by itself establish that the event caused a later performance difference. Establishing a plausible causal pathway requires following how the state difference is propagated through the subsequent execution.

This distinction motivates the tracing used in the experiments. Aggregate solver statistics are used to measure the magnitude of the performance variation across permutations, while detailed execution traces are used to examine where equivalent runs begin to differ and what solver state is associated with that divergence. The aim is not merely to identify which representation is faster, but to understand how a representation-level perturbation can become a difference in computational trajectory.

3. Research Questions and Hypotheses

The experiment is designed to separate four related questions: whether variable renaming changes solver effort, when equivalent executions first become observably different, how any resulting difference propagates through the solver's internal state, and do large performance divergences arise through a common mechanism or through multiple divergence pathways.

3.1. Research Questions

RQ1: How much can semantically equivalent variable permutations change the computational effort of a deterministic CDCL solver?

This question establishes the magnitude and distribution of the effect. Solver effort is evaluated using the number of decisions, conflicts, propagations, and elapsed execution time across the generated permutations.

RQ2: At what point do equivalent solver executions first become observably different?

The purpose of this question is to distinguish an early divergence in the search trajectory from a difference that only becomes visible after a substantial portion of the execution has already occurred.

RQ3: Through what changes in solver state can a representation-level perturbation propagate into a substantially different search trajectory?

This question focuses on the mechanism underlying the observed performance differences. In particular, the analysis examines branching decisions, variable activity, branching order, conflicts, and other observable state changes that may connect the initial representation difference to later search behavior.

RQ4: Do large performance divergences arise through a common mechanism, or through multiple divergence pathways?

A single permutation may provide evidence for one causal pathway without explaining the behavior of other permutations. This question therefore tests whether large accelerations and slowdowns can be characterized by one general pattern or whether different executions diverge through different sequences of state transitions.

3.2. Hypotheses

The experiments are guided by four corresponding hypotheses.

H1: Semantically equivalent variable permutations will produce measurable variation in solver effort despite preserving the underlying Boolean problem.

H2: At least some large performance differences will be preceded by an observable divergence in the solver's branching or heuristic state.

H3: An early difference in solver state can be amplified through subsequent decisions, conflicts, learning, and heuristic updates, producing a substantially different search trajectory.

H4: Large performance divergences will not necessarily share a single immediate cause; different permutations may exhibit distinct pathways to trajectory divergence.

4. Experimental Methodology

4.1. Solver and Configuration

All experiments were conducted using MiniSat version 2.2.0, a CDCL SAT solver designed around a compact and extensible implementation of the CDCL framework. A fixed solver configuration was used throughout the experiment so that differences between runs arose from the input representation rather than from changes in solver parameters.

The solver was compiled locally from the MiniSat source code using the same build for every experiment. No randomization or per-instance parameter changes were introduced between runs. Each CNF instance was solved from an otherwise fresh solver state, preventing heuristic information, learned clauses, or assignments from one run from carrying over into another.

The solver was instrumented during the mechanistic analysis to expose additional execution information that is not included in the default solver output. In particular, the instrumentation records individual

branching decisions, selected variable identities, variable activity values, and positions of selected variables within the branching priority structure. These modifications were used only for the detailed trace experiments and did not alter the decision procedure itself.

For every run, the solver recorded four primary measures of computational effort: the total number of branching decisions, the total number of conflicts, the total number of propagations, and elapsed execution time. These measures capture different aspects of the search. Decisions represent the number of explicit branching choices made by the solver, conflicts measure the number of contradictions encountered during search, and propagations measure the deductions performed as a consequence of assignments. Elapsed time provides an overall runtime measure but is interpreted alongside the solver-level counters because execution time can also be affected by factors outside the logical search process.

All runs used the same hardware and software environment. The only intentional difference between the baseline instance and its experimental variants was the permutation of variable identities described in the following section.

4.2. Benchmark Instances

The performance experiment uses five SAT instances drawn from the public ROCSAT benchmark corpus. The instances were selected to provide non-identical problem structures while remaining practical for repeated execution under the experimental MiniSat configuration. The benchmark set comprises two bounded-model-checking instances, `bmc-ibm-12.cnf` and `bmc-ibm-13.cnf`; the crafted UNSAT instance `aim-100-1_6-no-1.cnf`; the Boolean vector synthesis instance `3bitadd_31.cnf`; and the graph-coloring instance `flat200-5.cnf`.

Each benchmark serves as a fixed base formula for its own permutation experiment. The benchmark formula itself is never regenerated or structurally modified. Instead, every experimental instance is derived directly from that formula by swapping the identities of two variables. This preserves the clause count, literal count, and logical relationships within each benchmark while changing only the concrete variable identifiers through which the solver encounters them.

The benchmarks differ substantially in their baseline search behavior. This variation is useful rather than incidental: the study is not intended to assume that all formulas respond identically to representational perturbation. Instead, the five instances provide a controlled comparison of whether the same transformation can produce different forms or magnitudes of trajectory sensitivity across problem structures.

The original, unpermuted execution of each benchmark is solved separately and provides the reference measurements for that benchmark. All subsequent permutations are compared with the corresponding unpermuted baseline rather than with a common absolute threshold. This normalization allows solver effort to be compared across instances whose intrinsic difficulties differ substantially.

4.3. Variable-Permutation Procedure

To isolate the effect of variable representation, experimental instances were generated from the same baseline CNF by applying a single swap of two variable identities. For a selected pair of variables x_i and x_j, every occurrence of x_i in the formula was replaced with x_j, and every occurrence of x_j was replaced with x_i. The signs of the literals were preserved during this operation. No clauses were added, removed, reordered, or otherwise modified.

Each swap therefore defines a bijective variable permutation. The resulting formula contains the same number of variables, clauses, and literals as the baseline instance, with the same clause-variable relationships up to the renaming of the variables. The transformed formula is consequently logically equivalent to the original instance under the corresponding renaming of assignments.

One hundred experimental instances were generated in this manner for each benchmark. For each instance, the two variables to be swapped were selected as a distinct pair from the variables appearing in

the corresponding baseline formula. The resulting files were named according to the identifiers of the swapped variables, allowing each experimental result to be traced back to its exact transformation.

The baseline formula was also solved separately without any permutation. This execution provides the reference trajectory and solver-effort measurements used throughout the analysis. Each permuted instance was then solved independently using the same MiniSat build and configuration.

The use of a single transposition rather than a completely randomized renaming keeps the perturbation deliberately small while preserving the logical equivalence of the transformed formula [5]. The experiments therefore ask whether changing only the identities of two variables is sufficient to alter the subsequent computational behavior of the solver, and how such a small representational change can develop into a difference in search effort.

4.4. Measured Metrics

The experiment records both aggregate measures of solver effort and execution-level information used for trajectory analysis.

The primary measure is the number of *decisions*. A decision is an explicit branching choice made by the solver when selecting an unassigned variable and assigning it a truth value. This measure is particularly relevant to the experiment because it captures how extensively the solver branches through the search space.

The number of *conflicts* records how many contradictions the solver encounters during search. A conflict causes CDCL conflict analysis and can lead to the creation of a learned clause, making the conflict count a useful measure of how the search trajectory interacts with the constraint database.

The number of *propagations* records the assignments produced through Boolean constraint propagation. Propagations capture work that occurs between explicit branching decisions and can vary substantially even when the number of decisions is similar.

Finally, *elapsed execution time* is recorded as an overall measure of runtime. Because wall-clock measurements can be affected by factors outside the logical search itself, runtime is interpreted together with the solver's internal counters rather than used as the sole measure of computational effort.

For selected permutations, the solver was additionally instrumented to record the sequence of branching decisions and selected internal heuristic state. These traces include the identity of the variable selected at each decision, its associated activity values when relevant, and its position within the variable-selection priority structure. Conflict events and their corresponding search state were also recorded for selected portions of the execution.

These measurements serve different purposes. Aggregate counters establish the magnitude of performance variation across the permutations, while the detailed traces are used to examine the relationship between internal state, branching decisions, and subsequent trajectory divergence.

4.5. Experimental Procedure

The experiment was performed in three stages. First, each of the five original benchmark instances was solved using the fixed MiniSat build and configuration described above. The resulting decision, conflict, propagation, and runtime measurements were recorded as the baseline against which the corresponding permuted instances were compared.

Second, 100 derived instances were generated from each baseline formula. Each derived instance was produced by selecting one pair of variables and swapping their identities throughout the formula. Every generated instance was then solved independently from a fresh solver state using the same configuration as its corresponding baseline run. The resulting solver statistics were collected into a common experimental record.

Third, a subset of permutations was selected for detailed trajectory analysis based on their substantially different solver effort. These runs were repeated with the solver instrumentation enabled. The additional

traces record branching decisions and selected heuristic state during the execution, allowing the behavior of individual runs to be compared with their corresponding baseline run at a finer level.

The baseline and permuted executions were not resumed from one another and did not share learned clauses, assignments, or heuristic state. Each run therefore represents an independent execution beginning from its own input representation.

For the trajectory analysis, executions were compared incrementally rather than only at termination. Decision sequences, conflict events, and relevant heuristic-state observations were aligned by their execution order to identify points at which the runs first became observably different. Cases in which the executions remained aligned for an extended period were retained as evidence of delayed divergence rather than being excluded as uninteresting.

This procedure separates the broad measurement of permutation sensitivity from the mechanistic analysis of individual trajectories. The 100-run experiment for each benchmark establishes how solver effort varies across equivalent representations, while the traced subset is used to investigate how those differences develop during search.

5. Results

5.1. Variation in Solver Effort

The 100 variable-swapped instances for each of the five benchmarks produced substantial variation in solver effort despite all instances within each benchmark being obtained from the same baseline formula by a single transposition of two variable identities. Every permuted instance preserved the logical problem of its corresponding baseline, and all 500 experimental runs completed successfully.

The magnitude and distribution of this variation differed across the five benchmarks. The baseline values and the observed ranges are summarized in Table 1.

Table 1. Solver-effort distributions across the five benchmark sets.

Benchmark	Metric	Minimum	Median	Mean	Maximum	Baseline
BMC-13	Decisions	2,845	49,619.5	44,770.0	62,320	49,726
	Conflicts	557	24,888	21,890.0	34,304	25,335
	Propagations	329,783	17.43M	15.33M	26.42M	18.39M
BMC-12	Decisions	23,859	40,028.5	37,027.4	46,362	40,692
	Conflicts	5,405	10,209.5	9,618.9	12,297	11,387
	Propagations	8.70M	17.50M	16.09M	20.79M	19.25M
AIM	Decisions	89	166	164.02	214	166
	Conflicts	23	25	25.55	32	24
	Propagations	285	384	385.44	518	384
3bitadd-31	Decisions	17,406	60,037.5	76,756.84	191,336	47,505
	Conflicts	1,206	10,198	16,052.36	48,741	8,635
	Propagations	213,322	1.988M	2.940M	9.419M	1.479M
flat200-5	Decisions	549	4,679	4,363.43	7,143	4,679
	Conflicts	376	3,775	3,505.05	5,761	3,775
	Propagations	46,057	492,671	457,773.97	756,672	492,671

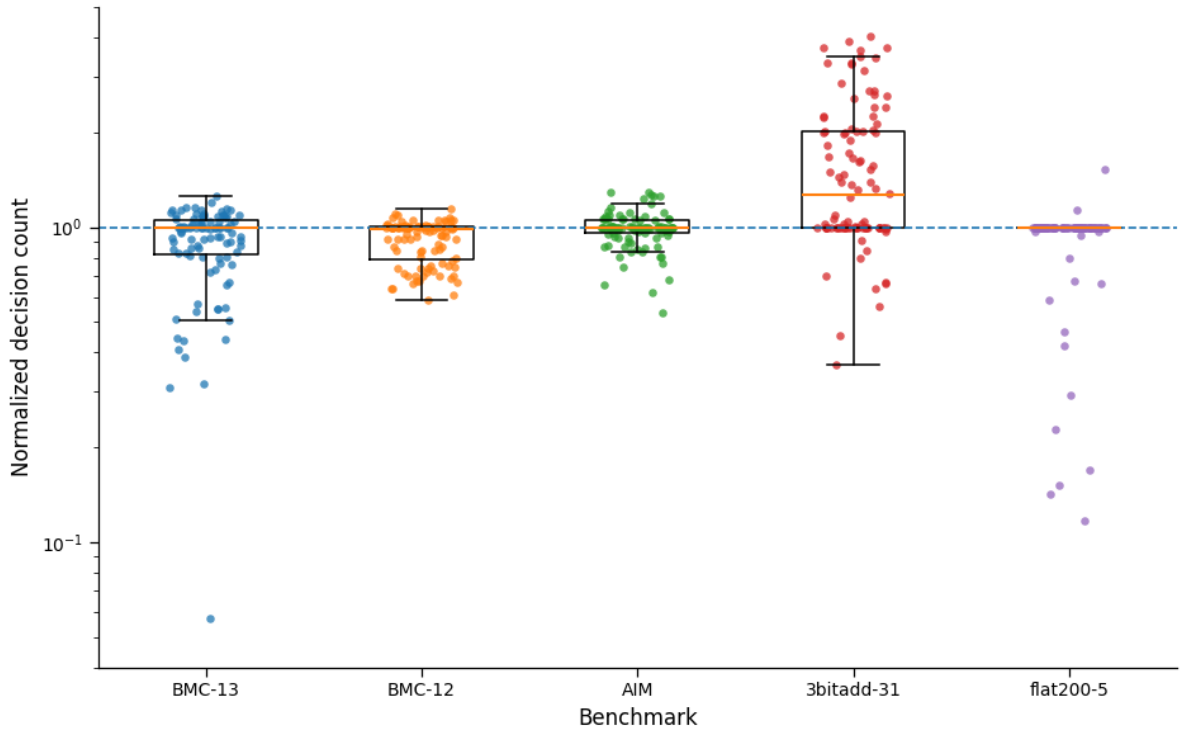


Fig. 1. Distribution of solver decisions across the 100 single-variable transpositions for each benchmark. The distributions show that representation sensitivity varies substantially between instances, with different benchmarks exhibiting different ranges, central tendencies, and degrees of outlier behavior.

The median decision counts of the permutation sets provide a useful indication of how closely a typical permutation remains to the original representation. For BMC-13, the median decision count of 49,619.5 is almost identical to the baseline value of 49,726, indicating that the typical permutation does not move the execution dramatically away from the original search effort. A similar pattern appears for BMC-12, whose median is 40,028.5 compared with a baseline of 40,692.

The other benchmarks exhibit different distributions. The median decision count for 3bitadd-31 is approximately 1.26 times the baseline, and the distribution extends substantially farther toward higher search effort than the BMC instances. For flat200-5, the median remains equal to the baseline despite the presence of permutations requiring substantially less or more search. AIM likewise remains relatively close to its baseline in absolute terms, although its smaller scale makes the observed variation proportionally more significant.

The extremes illustrate the scale of the effect more clearly than the central tendency. On BMC-13, the fastest observed permutation was `swap_5787_9740.cnf`. It required only 2,845 decisions, compared with 49,726 for the baseline. The same run required only 557 conflicts and 329,783 propagations. The opposite behavior is observed for `swap_917_10387.cnf`, which required 62,320 decisions, 34,304 conflicts, and 26,420,880 propagations. These cases remain the most extreme examples of two-sided variation in a single benchmark.

A similarly large deviation occurs for 3bitadd-31. Its fastest observed permutation required 17,406 decisions, while its slowest required 191,336. The slowest case therefore performed more than four times as many decisions as the baseline. In contrast, flat200-5 was comparatively stable for most permutations, but still contained a permutation requiring only 549 decisions, approximately one ninth of the baseline search.

The existence of these different distributions is important. The effect is not well described as a general tendency for variable renaming to make the problem harder or easier. Instead, different benchmark instances exhibit different degrees and directions of sensitivity to the same representational perturbation.

The corresponding ratios make the asymmetry especially clear. The fastest permutation performed approximately 17.5 times fewer decisions, 45.5 times fewer conflicts, and 55.8 times fewer propagations than the baseline. The slowest permutation, in contrast, required approximately 1.25 times as many decisions, 1.35 times as many conflicts, and 1.44 times as many propagations.

The effect is therefore not well described as a general tendency for variable renaming to make the instance harder or easier. Most permutations remain relatively close to the original search effort, but a subset of permutations move far from the baseline, and they can do so in either direction. The distribution is consequently characterized by substantial outlier behavior rather than a uniform shift.

This can also be seen by comparing the four metrics with one another. The extreme cases are not isolated runtime anomalies: the permutation that requires dramatically fewer decisions also requires dramatically fewer conflicts and propagations, while the slowest permutation increases all three search counters simultaneously. The representational perturbation is therefore associated with a change in the amount of search performed by the solver itself.

At the same time, the aggregate measurements do not reveal the mechanism responsible for these changes. In particular, they cannot determine whether the divergence originated at the first branching decision, accumulated through heuristic updates, appeared only after a later conflict, or resulted from a combination of these effects. The next sections therefore move from the distribution of solver effort to individual executions and their search trajectories.

The corresponding ratios make the asymmetry especially clear. The fastest permutation performed approximately 17.5 times fewer decisions, 45.5 times fewer conflicts, and 55.8 times fewer propagations than the baseline. The slowest permutation, in contrast, required approximately 1.25 times as many decisions, 1.35 times as many conflicts, and 1.44 times as many propagations.

The effect is therefore not well described as a general tendency for variable renaming to make the instance harder or easier. Most permutations remain relatively close to the original search effort, but a subset of permutations move far from the baseline, and they can do so in either direction. The distribution is consequently characterized by substantial outlier behavior rather than a uniform shift.

This can also be seen by comparing the four metrics with one another. The extreme cases are not isolated runtime anomalies: the permutation that requires dramatically fewer decisions also requires dramatically fewer conflicts and propagations, while the slowest permutation increases all three search counters simultaneously. The representational perturbation is therefore associated with a change in the amount of search performed by the solver itself.

At the same time, the aggregate measurements do not reveal the mechanism responsible for these changes. In particular, they cannot determine whether the divergence originated at the first branching decision, accumulated through heuristic updates, appeared only after a later conflict, or resulted from a combination of these effects. The next sections therefore move from the distribution of solver effort to individual executions and their search trajectories.

5.2. Benchmark-Specific Outcomes

5.2.1. BMC-12

The `bmc-ibm-12.cnf` instance showed a more moderate response to variable transposition than BMC-13, although the permutations still produced measurable changes in solver effort. The baseline execution required 40,692 decisions, while the 100 single-variable transpositions produced decision counts ranging from 23,859 to 46,362. The median was 40,028.5 decisions, and the mean was 37,027.4, placing the typical permutation relatively close to the baseline. In total, 52 of the 100 permutations required fewer decisions than the baseline, 32 required more, and 16 produced the same decision count.

The variation was also visible in the other solver counters. Conflicts ranged from 5,405 to 12,297, compared with 11,387 for the baseline, while propagations ranged from approximately 8.70 million to 20.79 million, compared with 19.25 million for the baseline. The fastest observed permutation, `swap_5626_30198`,

required 23,859 decisions and 5,405 conflicts, whereas the slowest, `swap_23109_35542`, required 46,362 decisions. Thus, although BMC-12 did not exhibit the extreme bidirectional variation observed for BMC-13, changing only two variable identities was still sufficient to alter the amount of search performed by the solver.

The distribution therefore suggests a moderate sensitivity regime: most permutations remained within a relatively narrow region around the baseline, but individual transpositions could still produce substantial reductions or increases in search effort. This contrasts with BMC-13 and demonstrates that the magnitude of representation sensitivity depends on the interaction between the particular instance and the solver's internal search process.

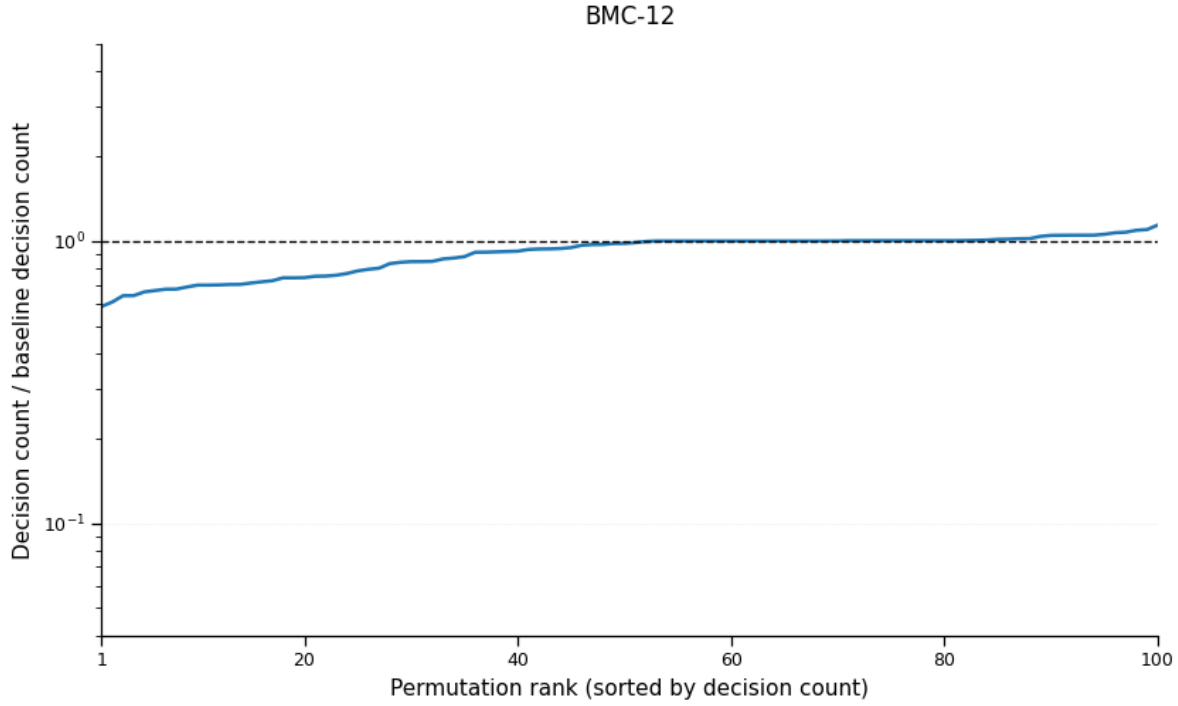


Fig. 2. Decision counts for the 100 single-variable transpositions of `bmc-ibm-12.cnf`, sorted from lowest to highest solver effort. The dashed horizontal line indicates the baseline execution.

5.2.2. BMC-13

The `bmc-ibm-13.cnf` instance exhibited the strongest two-sided sensitivity among the five benchmarks. The baseline execution required 49,726 decisions, while the 100 single-variable transpositions produced decision counts ranging from 2,845 to 62,320. The median permutation required 49,619.5 decisions, remaining almost identical to the baseline, but the distribution contained substantial deviations in both directions. The fastest observed permutation, `swap_5787_9740`, required only 2,845 decisions, whereas the slowest, `swap_917_10387`, required 62,320 decisions. Thus, the fastest permutation used approximately 17.5 times fewer decisions than the slowest.

The same pattern appeared in the other internal solver counters. The fastest permutation required only 557 conflicts and 329,783 propagations, compared with 25,335 conflicts and 18,392,775 propagations for the baseline. Conversely, the slowest permutation required 34,304 conflicts and 26,420,880 propagations. The fact that decisions, conflicts, and propagations change together indicates that the observed variation reflects substantially different amounts of search rather than an isolated difference in elapsed runtime.

The distribution is therefore characterized by a relatively typical central region combined with extreme outliers in both directions. Most permutations remain comparatively close to the baseline, but a small number cause the solver to enter trajectories requiring dramatically less or more search. This makes

BMC-13 particularly useful for the subsequent mechanistic analysis, since it contains both a strongly accelerated execution and a strongly slowed execution produced by the same type of representational perturbation.

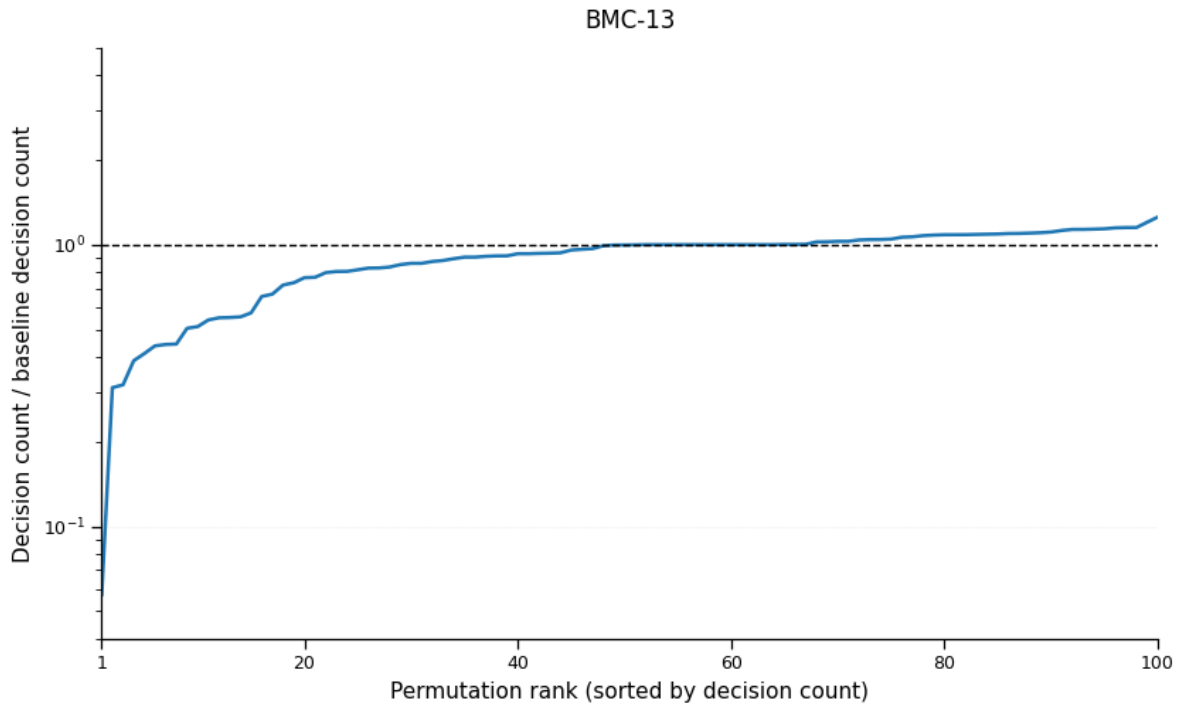


Fig. 3. Decision counts for the 100 single-variable transpositions of `bmc-ibm-13.cnf`, sorted from lowest to highest solver effort. The dashed horizontal line indicates the baseline execution.

5.2.3. AIM-100-1

The `aim-100-1_6-no-1.cnf` instance exhibited comparatively limited sensitivity to variable transposition. The baseline execution required 166 decisions, 24 conflicts, and 384 propagations. Across the recorded valid permutation runs, decision counts ranged from 89 to 214, while conflicts ranged from 23 to 32 and propagations from 285 to 518. The median decision count remained 166, equal to the baseline, and the mean was approximately 164.02. These results indicate that most permutations remained relatively close to the original execution, although individual swaps were still capable of producing noticeable proportional changes given the small absolute size of the instance.

Unlike BMC-13 and BMC-12, the AIM results do not contain very large absolute changes in solver effort. The narrow conflict and propagation ranges further indicate that the effect of variable renaming was comparatively constrained for this instance. Nevertheless, the presence of permutations requiring only 89 decisions as well as others requiring 214 shows that semantic equivalence did not produce completely identical computational behavior even in this relatively stable case.

The AIM results therefore provide a contrasting sensitivity regime within the experiment. Whereas some benchmarks exhibit broad distributions or substantial high-effort tails, AIM remains concentrated around its baseline. This supports the conclusion that representation sensitivity is strongly instance-dependent rather than an invariant consequence of applying a variable transposition.

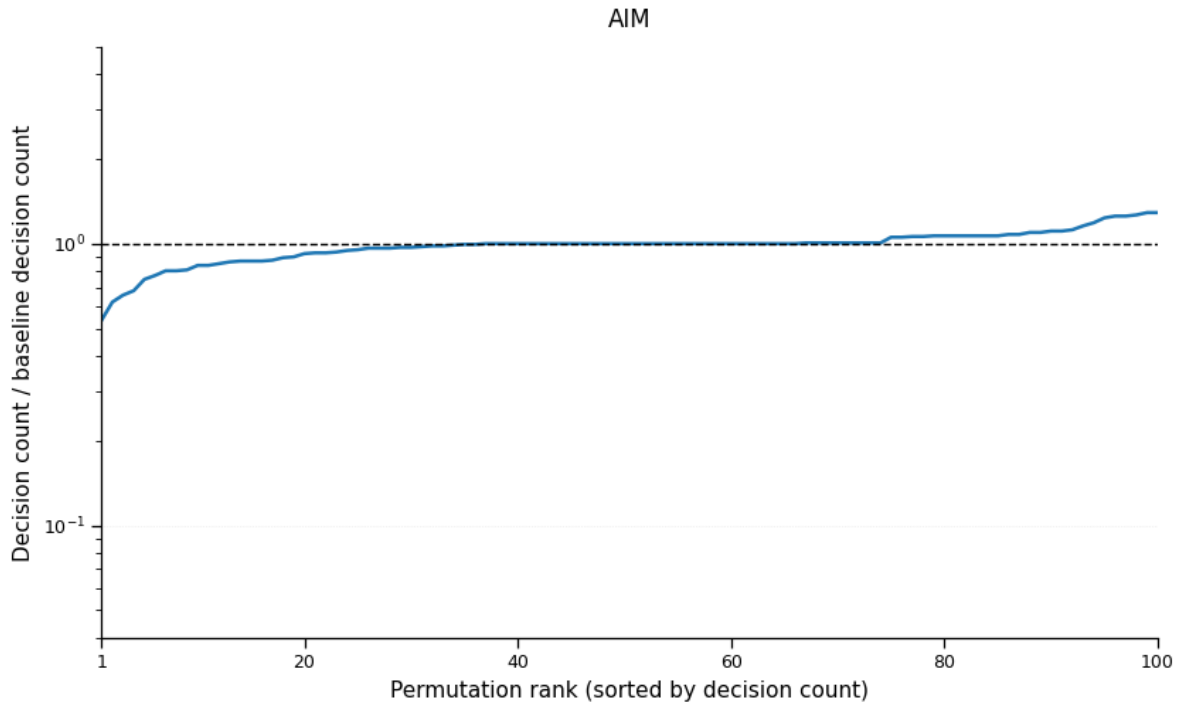


Fig. 4. Decision counts for the recorded single-variable transpositions of `aim-100-1_6-no-1.cnf`, sorted from lowest to highest solver effort. The dashed horizontal line indicates the baseline execution.

5.2.4. 3bitadd-31

The `3bitadd_31.cnf` instance exhibited a substantially different sensitivity pattern from the two BMC instances. The baseline execution required 47,505 decisions, while the 100 single-variable transpositions produced decision counts ranging from 17,406 to 191,336. The median was 60,037.5 decisions and the mean was 76,756.84, both higher than the baseline. In total, 60 of the 100 permutations required more decisions than the baseline, 12 required fewer, and 28 produced the same decision count. The distribution therefore shows a clear shift toward higher search effort rather than the roughly balanced behavior observed for BMC-13 and BMC-12.

The same pattern was visible in the other search counters. Conflicts ranged from 1,206 to 48,741, compared with 8,635 for the baseline, while propagations ranged from 213,322 to 9.419 million, compared with 1.479 million for the baseline. The minimum and maximum decision counts correspond to approximately 0.37 and 4.03 times the baseline, respectively. These results show that the high-effort tail is not an artifact of a single counter: permutations requiring more decisions also produced substantially more conflicts and propagations.

The distribution consequently represents a distinct sensitivity regime within the benchmark set. Rather than remaining centered near the original search effort, the permutation set is shifted toward larger decision counts and contains a pronounced high-effort tail. This demonstrates that a logically irrelevant representational change can interact with a particular SAT instance in a strongly asymmetric way, producing substantially more search for a majority of the tested transpositions.

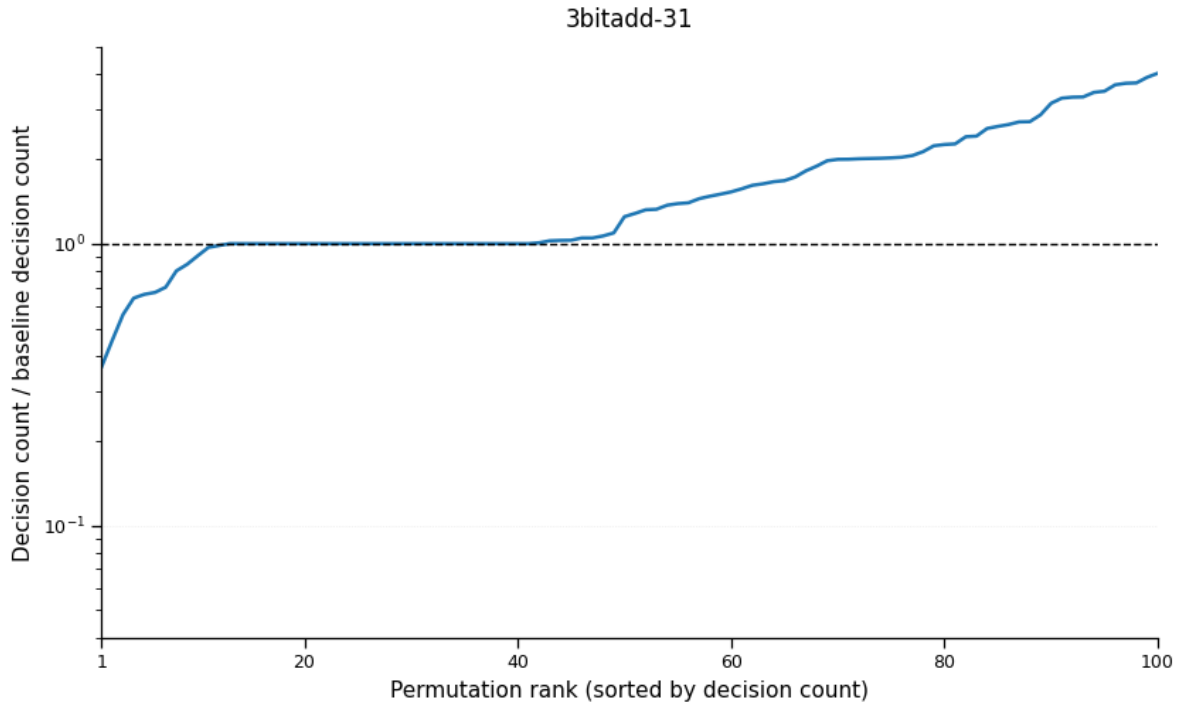


Fig. 5. Decision counts for the 100 single-variable transpositions of `3bitadd_31.cnf`, sorted from lowest to highest solver effort. The dashed horizontal line indicates the baseline execution.

5.2.5. flat200-5

The `flat200-5.cnf` instance was comparatively stable across most of the tested variable transpositions, although several permutations still produced substantial deviations from the baseline. The baseline execution required 4,679 decisions, 3,775 conflicts, and 492,671 propagations. Across the 100 single-variable transpositions, decision counts ranged from 549 to 7,143, with a median of 4,679 and a mean of 4,363.43. In total, 38 permutations required fewer decisions than the baseline, 15 required more, and 47 produced the same decision count. The median therefore remained exactly at the baseline even though the full range was considerably wider.

The other solver counters followed the same general pattern. Conflicts ranged from 376 to 5,761, while propagations ranged from 46,057 to 756,672. The fastest observed permutation required only 549 decisions, approximately 11.7% of the baseline, whereas the slowest required 7,143 decisions, approximately 1.53 times the baseline. The corresponding changes in conflicts and propagations show that these deviations represent changes in search effort rather than isolated differences in one measured quantity.

The resulting distribution differs from the strongly asymmetric behavior observed for 3bitadd-31 and the pronounced two-sided variation of BMC-13. Most permutations remain close to the original search effort, but a small number produce much smaller or larger executions. This makes flat200-5 an example of a comparatively stable benchmark in which representation sensitivity is present primarily through isolated outliers rather than a broad shift in the distribution.

5.3. Extreme Permutation Outcomes

Although the median performance of the permuted instances remains close to their corresponding baselines for several benchmarks, individual permutations produce substantially different search effort. These outliers are useful for the trajectory analysis because they demonstrate that the effect is not confined to small numerical fluctuations.

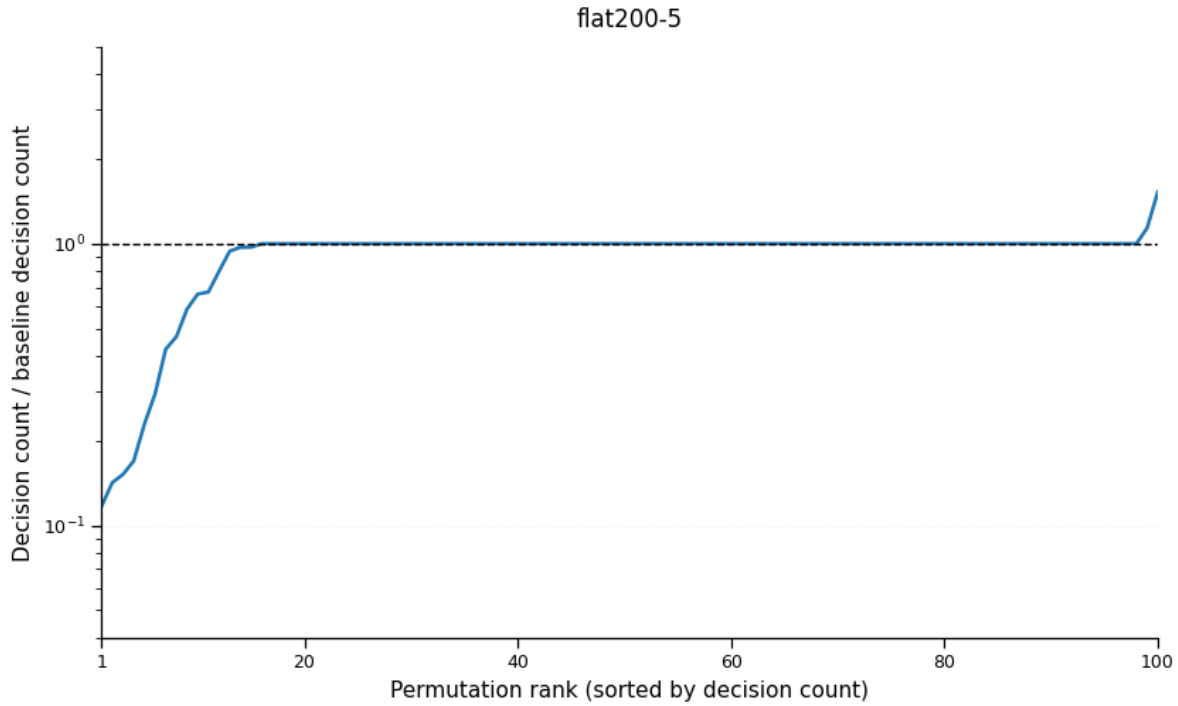


Fig. 6. Decision counts for the 100 single-variable transpositions of `flat200-5.cnf`, sorted from lowest to highest solver effort. The dashed horizontal line indicates the baseline execution.

The strongest acceleration observed in the study occurred for `bmc-ibm-13.cnf`. The permutation `swap_5787_9740.cnf` exchanges the identities of variables 5787 and 9740 and leaves all other variable identities unchanged. Despite this minimal modification, the resulting execution terminates after only 2,845 decisions, compared with 49,726 for the baseline. The reduction is accompanied by similarly large decreases in conflicts and propagations: 557 conflicts and 329,783 propagations are recorded, compared with 25,335 and 18,392,775 in the original execution.

The opposite behavior is observed for `swap_917_10387.cnf`, which exchanges variables 917 and 10387. This instance requires 62,320 decisions, 34,304 conflicts, and 26,420,880 propagations. Unlike the accelerated case, the permutation therefore pushes the solver into a trajectory involving substantially more search before a solution is reached.

Table 2 compares these two extreme permutations with the original representation.

Table 2. Baseline and extreme permutation outcomes for BMC-13.

Instance	Decisions	Conflicts	Propagations	Runtime (ms)
Baseline	49,726	25,335	18.39M	1,738.1
`swap_5787_9740`	2,845	557	0.330M	50.6
`swap_917_10387`	62,320	34,304	26.42M	2,458.8

The corresponding ratios make the asymmetry especially clear. The fastest permutation performed approximately 17.5 times fewer decisions, 45.5 times fewer conflicts, and 55.8 times fewer propagations than the baseline. The slowest permutation, in contrast, required approximately 1.25 times as many decisions, 1.35 times as many conflicts, and 1.44 times as many propagations.

The existence of both extremes within the same family of transformations is important. The experiment does not reveal a consistent relationship between the act of renaming variables and solver difficulty. Swapping two identities can result in either a dramatic reduction or a substantial increase in search effort.

The direction and magnitude of the effect depend on the particular permutation.

The distribution also contains several additional high-effort outliers. For example, `swap_5469_6439.cnf` requires 59,741 decisions and 33,395 conflicts, while `swap_1095_4289.cnf` requires 57,210 decisions and 31,186 conflicts. `swap_1791_5979.cnf` requires 57,180 decisions and 30,352 conflicts. These instances are sufficiently far from the baseline that their behavior cannot be attributed to the small differences in execution time expected from ordinary measurement noise.

At the same time, not every large outlier exhibits the same relationship between its early execution and its final performance. This becomes apparent when the executions are examined at the level of their decision and conflict sequences. In particular, some permutations show an identifiable difference very early in search, whereas others remain aligned with the baseline for a considerable number of conflicts before their eventual computational effort separates.

This observation determines how the remaining analysis is organized. The extreme permutations are not treated as interchangeable examples of the same phenomenon. Instead, representative executions are examined according to the point and nature of their divergence. The `swap_917_10387.cnf` execution provides an example of an early observable branching divergence, while `swap_1791_5979.cnf` provides an example in which substantial differences in final search effort do not correspond to an immediately visible difference in the recorded trajectory.

These cases provide complementary evidence. The first makes it possible to inspect a specific state difference that is followed almost immediately by a different search path. The second tests whether the same explanation can be applied to an outlier whose execution remains synchronized with the baseline for a much longer interval. The distinction is important because identifying one plausible divergence mechanism does not establish that it explains all permutations.

6. Mechanistic Analysis

6.1. Early Trajectory Divergence

The `swap_917_10387.cnf` permutation provides a clear example in which a small difference in the solver's internal ordering is followed by an immediate divergence in the search trajectory. The baseline and permuted executions remain closely aligned through the early portion of the search. In particular, conflicts 20, 21, and 22 occur at the same decision counts in both executions: 103, 112, and 114, respectively. Up to this point, the observable search histories therefore provide no indication of the large difference in total effort that eventually develops.

The first clear difference occurs at decision 123. At this point, the activities of variables 10694 and 10695 are identical in both executions:

$$\text{activity}(10694) = \text{activity}(10695) = 1.5866734416093373.$$

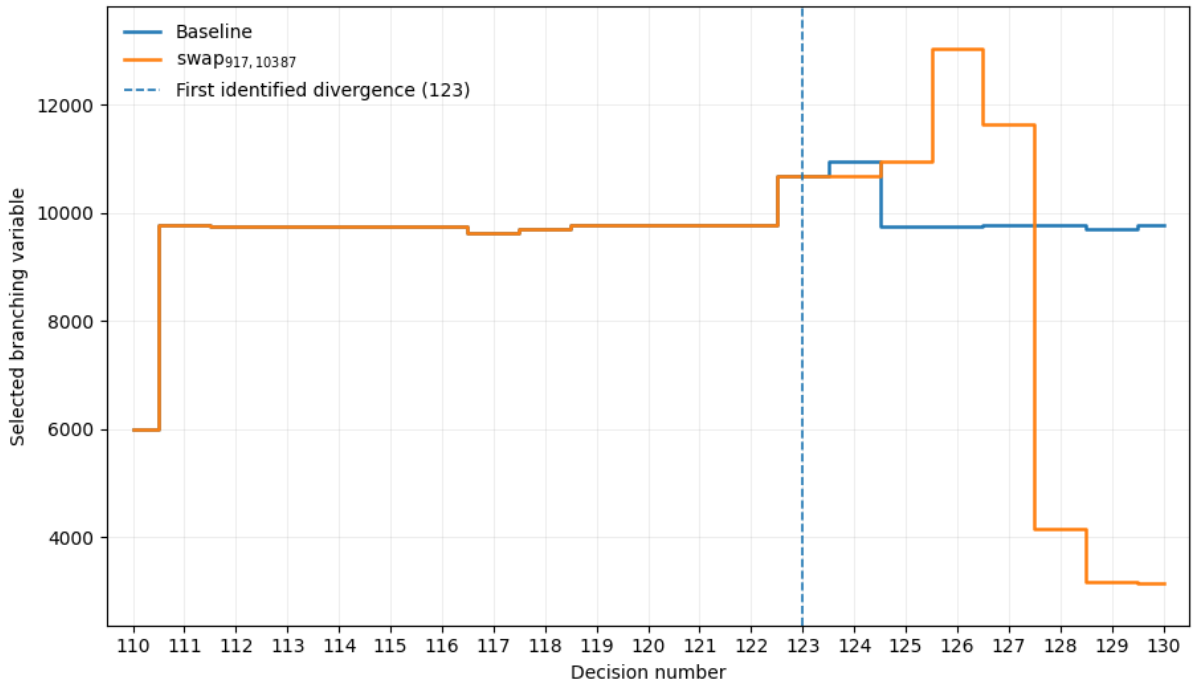


Fig. 7. Selected branching variables from decisions 110–130 for the baseline and `swap_917_10387` executions. The two executions follow the same recorded branching sequence through decision 122 and first differ at decision 123, where the baseline selects variable 10694 and the permuted execution selects variable 10695.

The difference is instead found in their positions within the branching priority structure. In the baseline execution, variable 10694 occupies position 18 while variable 10695 occupies position 37. The solver therefore selects variable 10694. In the permuted execution, these positions are reversed: variable 10694 is at position 37 and variable 10695 is at position 18, causing variable 10695 to be selected instead.

Table 3 summarizes the solver state at the first identified branching divergence for `swap_917_10387.cnf`.

Table 3. Solver state at the first identified branching divergence for `swap_917_10387.cnf`.

State variable	Baseline	`swap_917_10387`
Decision	123	123
Activity of 10694	1.5866734416093373	1.5866734416093373
Activity of 10695	1.5866734416093373	1.5866734416093373
Heap position of 10694	18	37
Heap position of 10695	37	18
Selected variable	10694	10695

This is notable because the branching decision cannot be attributed to a difference in the recorded activity values of the two candidates. At the moment of divergence, the numerical heuristic scores are equal; what differs is their ordering within the structure used by the solver to select a branching candidate. The variable permutation has therefore produced a difference in concrete solver state that is invisible if activity values alone are considered.

The consequences of the different branch appear immediately in the next conflict. In the baseline execution, conflict 23 occurs at decision 123, with the solver at decision level 35 before backtracking to level 26. The learned conflict clause contains the literals

$$10694,\ 10755,\ -10954,\ 10793.$$

In the permuted execution, conflict 23 occurs one decision later, at decision 124. The solver is at decision level 36 and backtracks only to level 35. The conflicting clause is instead

$$10694,\ 10754,\ 10793,\ -10954.$$

Thus, the different branch is followed not merely by a different variable selection but by a different conflict state and a different result of conflict analysis.

Table 4 gives the corresponding conflict-level comparison.

Table 4. First differing conflict between the baseline and `swap_917_10387`.

State variable	Baseline	`swap_917_10387`
Conflict	23	23
Decision count	123	124
Decision level	35	36
Backtrack level	26	35
Clause size	4	4
Conflict clause	10694, 10755, -10954, 10793	10694, 10754, 10793, -10954

Importantly, this chain begins without any difference in the recorded variable activities of the two competing candidates. The representational perturbation is therefore not acting by simply giving priority to one variable according to the heuristic score. Instead, it changes the concrete ordering used when the heuristic encounters tied candidates.

The importance of this sequence is that the first measured difference occurs in the solver's concrete internal state even though the corresponding heuristic scores are numerically identical. Once a different branch is selected, the resulting conflict analysis produces a different learned clause, after which the subsequent solver state is no longer constrained to follow the same trajectory as the baseline.

The large final performance difference for this permutation is therefore consistent with a path-dependent amplification of a small representational perturbation. The original and renamed instances encode the same Boolean problem, but the deterministic solver does not operate solely on that abstract problem. It operates on a concrete representation whose variable identities participate in internal ordering decisions. In this case, that ordering difference is sufficient to alter the subsequent search path.

This case provides evidence for an early divergence pathway from representation to search trajectory. It does not, however, establish that the same mechanism explains every large performance difference in the experiment. The remaining question is whether other extreme permutations also become distinguishable through an early branching event, or whether a substantially different form of divergence can occur later in the search.

6.2. Delayed Trajectory Divergence

The early divergence observed in `swap_917_10387` does not provide a complete explanation for the behavior of the other large outliers. In particular, `swap_1791_5979` requires 57,180 decisions and 30,352 conflicts, yet its recorded conflict trajectory remains aligned with the baseline through the first 300 conflicts examined.

For `swap_1791_5979`, the first part of the execution therefore does not exhibit the same immediate branching difference identified in the previous case. The permutation is already present in the input, but the recorded conflict sequence continues to coincide with that of the original representation for a substantial portion of the search. This is important because it demonstrates that a large final performance difference does not necessarily imply an early, obvious difference in the sequence of conflicts.

The same behavior is observed for another high-effort permutation, `swap_5469_6439`. Its final

execution requires 59,741 decisions and 33,395 conflicts, substantially more than the baseline, while the observed conflict trace remains aligned with the baseline over the corresponding portion of the analysis.

This creates a different form of trajectory divergence from the one observed for `swap_917_10387`. In the latter case, a different branching choice is visible at decision 123 and is immediately followed by a different conflict. In the delayed cases, the available trace does not expose such an early break in the conflict sequence. The executions can therefore remain observationally synchronized at the level of recorded conflicts even though their eventual computational effort is substantially different.

The distinction is particularly clear in the decision-level traces. For the baseline and `swap_1791_5979`, the recorded decision sequences in the traced interval from decision 1191 through decision 2000 are identical. The final solver statistics nevertheless differ substantially: the baseline terminates after 49,726 decisions, whereas `swap_1791_5979` requires 57,180.

This result places an important limitation on any attempt to identify the cause of representation sensitivity solely by comparing the first visible difference in branching decisions. A pair of executions can remain identical under the particular observables being recorded while still accumulating differences elsewhere in their internal state. Alternatively, the computations may genuinely remain equivalent for that interval and diverge only later. The available traces cannot distinguish these possibilities by themselves.

The delayed cases therefore provide a useful counterpoint to the early divergence example. They show that the existence of a large performance gap does not imply that a corresponding early difference will be visible in every monitored quantity. In particular, conflict and decision traces should be treated as observations of the solver trajectory rather than as a complete representation of solver state.

Taken together with the early-divergence case, these results suggest that representation sensitivity cannot be reduced to a single rule such as "renaming changes the first branching decision." One execution exposes a clear early ordering difference, while other high-effort executions remain aligned under the recorded observations for a substantial portion of their search. Understanding the broader phenomenon therefore requires considering multiple possible pathways through which differences in concrete solver state can eventually become differences in computational effort.

6.3. Evidence for Multiple Divergence Pathways

The two trajectory analyses do not support a single universal mechanism for the observed performance differences. The `swap_917_10387` execution provides a case in which an identifiable difference in branching structure is followed almost immediately by a different decision and conflict trajectory. The `swap_1791_5979` and `swap_5469_6439` executions, in contrast, remain aligned with the baseline under the recorded decision and conflict observations for substantial portions of the search despite eventually requiring considerably more work.

These cases differ in more than the magnitude of their final performance gap. They differ in the point at which that gap becomes visible and in the kind of observable state involved when the executions cease to align. This makes it difficult to describe representation sensitivity using a single chain of events that begins with the first decision and then deterministically produces the final outcome.

A useful distinction is therefore between an early divergence pathway and a delayed divergence pathway. In the early case, a difference in the ordering of equally active branching candidates is observable at a specific decision. That difference immediately changes the selected variable, after which the two executions encounter different conflict states. The causal sequence is comparatively easy to localize because the changes occur in close succession.

The delayed cases do not provide an equivalent localized sequence in the recorded traces. The executions can follow the same observed decisions and conflict progression for a substantial interval even though their final decision counts differ by thousands. This means that the eventual performance difference cannot be inferred simply by examining the first place at which the recorded branching sequence differs. Either the relevant state difference is not represented by the quantities being traced, or the two executions genuinely remain sufficiently similar until a later event causes the trajectories to separate.

The distinction also places a limit on the interpretation of the heap-order observation. The reversed positions of variables 10694 and 10695 provide strong evidence for one mechanism in the `swap_917_10387` case, but the existence of other high-effort permutations that do not exhibit the same early observable behavior prevents that mechanism from being treated as a general explanation for all slowdowns.

The broader result is therefore better characterized as path dependence than as a single implementation-level defect. The variable permutation creates a different concrete representation of the same logical instance. That difference can interact with the solver's heuristic and data structures at different points in the computation. Once the resulting state difference affects a future transition, the new transition changes the state available to all subsequent transitions, allowing the initial perturbation to become increasingly separated from its eventual computational consequences.

The experiments consequently distinguish between identifying a mechanism that can produce divergence and identifying a mechanism that explains every divergence. The former is demonstrated by the early case; the latter is not supported by the observed traces. Instead, the evidence indicates that semantically equivalent representations can enter substantially different computational histories through more than one observable pathway.

7. Discussion

The experiments show that semantic equivalence does not imply computational equivalence for the deterministic CDCL solver examined in this work. A single transposition of two variable identities was sufficient to produce outcomes ranging from 2,845 decisions to 62,320 decisions on the same underlying SAT instance. The important observation is not simply the size of this range, but the fact that the direction of the effect is not determined by the transformation itself. The same kind of representational perturbation can lead to substantially less or substantially more search.

This behavior is consistent with a stateful view of heuristic computation. The Boolean formula determines the constraints that the solver must satisfy, but the execution also depends on the concrete state maintained while those constraints are processed. Variable activities, branching order, assignments, learned clauses, and other internal structures participate in determining which transition occurs next. A change to the representation can therefore affect the execution even when it leaves the underlying Boolean problem unchanged.

The cross-benchmark results reinforce this interpretation. The five benchmark families do not exhibit a uniform sensitivity pattern. `bmc-ibm-13.cnf` shows particularly strong bidirectional variation, with permutations producing both dramatic speedups and substantial slowdowns. `bmc-ibm-12.cnf` shows a more moderate range of variation, while `aim-100-1_6-no-1.cnf` exhibits comparatively limited changes in solver effort. The `3bitadd_31.cnf` benchmark shows a pronounced shift toward higher decision counts across the permutation set, with a long high-effort tail, whereas `flat200-5.cnf` is comparatively stable for many permutations but still contains large deviations from the baseline. These differences indicate that representation sensitivity is strongly instance-dependent rather than being a fixed property of variable renaming alone.

The direction of the effect is also not reliably predictable from the size or location of the swap. In the BMC-13 experiments, swapping only two variable identities can reduce the number of decisions from 49,726 to 2,845 or increase it to 62,320. The transformation applied to each instance is structurally identical: exactly two variable identities are exchanged, with the clause structure otherwise preserved. The resulting solver behavior nevertheless varies substantially. This suggests that the relevant quantity is not the syntactic magnitude of the transformation, but how the transformed representation interacts with the solver's current and future internal state.

The mechanistic analysis gives a concrete example of how such amplification can occur. For `swap_917_10387`, the first identified branching difference occurs when variables 10694 and 10695 have exactly equal recorded activity values. Their heap positions are nevertheless reversed between the two

executions, causing different variables to be selected. The next conflict then occurs at a different decision count, at a different decision level, with a different backtrack level and a different conflict clause. Thus, a representational difference that is not visible in the numerical heuristic scores can become a branching difference through the concrete ordering maintained by the solver.

At the same time, the delayed-divergence cases show why this mechanism cannot be treated as a complete explanation of the phenomenon. The `swap_1791_5979` and `swap_5469_6439` executions eventually require substantially more work than the baseline, yet their recorded conflict trajectories remain aligned with the baseline over substantial portions of the observed search. In the case of `swap_1791_5979`, the decision traces are also identical over the traced interval from decisions 1191 through 2000. The available measurements therefore do not reveal a single early event analogous to the heap-order difference observed for `swap_917_10387`.

This distinction is important for interpreting the central phenomenon. The results support a causal mechanism for at least one observed divergence, but they do not support the stronger claim that one implementation-level rule explains every permutation. The evidence instead points toward path dependence in which small differences in concrete solver state can interact with a sequence of heuristic and data-structure transitions. Once a state difference affects a later transition, the resulting state can alter subsequent transitions, allowing a small initial perturbation to become a large difference in total computational effort.

The findings therefore suggest that the computational behavior of a deterministic SAT solver is partly a property of the representation through which the problem is presented. Semantic equivalence determines which assignments solve the instance, but it does not uniquely determine the execution history of a stateful heuristic search procedure. Equivalent representations can induce different internal orderings, different branches, different learned clauses, and ultimately different amounts of computation.

The results should not be interpreted as evidence that variable renaming universally makes SAT solving harder or easier. Rather, they demonstrate that a logically irrelevant transformation can be computationally consequential, and that the resulting sensitivity can range from negligible to extreme depending on the interaction between the representation, the benchmark instance, and the solver's internal state.

Overall, the experiments support the view that solver performance should be understood not only in terms of the logical structure of the input formula, but also in terms of the computational trajectory induced by a particular representation. The same Boolean problem can therefore correspond to markedly different execution histories under a deterministic CDCL procedure, even when the only intentional change is the renaming of two variables.

8. Limitations

The scope of the experiment places several limits on the conclusions that can be drawn from the results.

First, the performance experiment uses five benchmark instances: `bmc-ibm-13.cnf`, `bmc-ibm-12.cnf`, `aim-100-1_6-no-1.cnf`, `3bitadd_31.cnf`, and `flat200-5.cnf`. This provides stronger evidence that representation sensitivity is not confined to a single formula, since the five instances exhibit substantially different sensitivity patterns, but it also limits the extent to which the observed behavior can be generalized to other SAT instances. Different formulas may interact with variable ordering and CDCL heuristics in substantially different ways.

Second, the experiments use a single solver implementation and configuration, MiniSat 2.2.0. The mechanisms observed here therefore describe behavior within this particular implementation rather than establishing that the same internal pathways occur in other CDCL solvers. Solver-specific data structures and heuristic policies may produce different forms of representation sensitivity.

Third, the mechanistic analysis observes only selected portions of the solver's internal state. The instrumentation records branching decisions, selected heuristic activity values, priority-structure positions,

and conflict information, but it does not capture every mutable object maintained by the solver. In particular, an execution may contain an internal difference that is not visible in the recorded trace. Consequently, the first observed divergence cannot always be identified with the first causal difference in the complete solver state.

Fourth, the experiment uses 100 single-variable transpositions for each benchmark rather than an exhaustive set of possible permutations. The results therefore characterize the behavior of controlled samples of small representational perturbations, not the full space of possible variable renamings.

Finally, elapsed execution time is inherently more sensitive to the execution environment than the solver's internal counters. Runtime measurements are therefore treated as supporting evidence rather than as the primary basis for the mechanistic conclusions. Decisions, conflicts, propagations, and execution traces provide more direct evidence about the search performed by the solver itself.

The benchmark set also contains instances with different observed sensitivity regimes, including cases dominated by acceleration, cases with substantial slowdowns, and cases in which most permutations remain comparatively close to the baseline while a small number produce large deviations. These differences strengthen the conclusion that the phenomenon is instance-dependent, but they also mean that the five selected benchmarks should not be treated as statistically representative of the broader SAT benchmark space.

The mechanistic conclusions are further constrained by the fact that the trace analysis focuses on selected representative permutations rather than every one of the 500 generated permutation cases. The `swap_917_10387` execution provides direct evidence for an early heap-order divergence, while the `swap_1791_5979` and `swap_5469_6439` executions demonstrate that substantial performance differences can remain hidden under the recorded observables for extended portions of the search. These observations establish possible divergence pathways, but they do not determine the complete internal cause of every observed performance difference.

These limitations do not undermine the central observation that equivalent representations can produce substantially different computational behavior in the tested setting. They instead define the scope of that observation and identify the extent to which further experiments would be required to establish broader generality.

9. Conclusion

A change in variable naming should not change a Boolean problem. Yet, in the experiments presented here, swapping the identities of only two variables was enough to produce dramatically different executions of a deterministic CDCL solver.

Across the five benchmark instances and their controlled single-variable transpositions, the experiments show that semantic equivalence of inputs does not guarantee computational equivalence. The magnitude and direction of the effect varied substantially between benchmarks. BMC-13 exhibited particularly strong two-sided variation, with the fastest observed permutation requiring only 2,845 decisions compared with 49,726 for the baseline, while another required 62,320 decisions. The 3bitadd-31 benchmark likewise produced a large high-effort tail, with observed decision counts ranging from 17,406 to 191,336. Other benchmarks, including BMC-12, AIM, and flat200-5, showed more moderate or more concentrated distributions, demonstrating that representation sensitivity is strongly dependent on the interaction between the particular formula and the solver.

The mechanistic analysis further shows that the effect can arise from internal solver state rather than from any change to the underlying Boolean problem. For `swap_917_10387`, variables 10694 and 10695 had identical recorded activity values at the first identified branching divergence, yet their positions in the branching priority structure were reversed. This changed the selected variable and was immediately followed by a different conflict state, backtrack level, and learned clause. The observed sequence provides concrete evidence that a representational perturbation can enter the solver through an internal ordering

difference and then propagate into a different search trajectory.

At the same time, the delayed-divergence cases demonstrate that this mechanism is not universal. The `swap_1791_5979` and `swap_5469_6439` executions remained aligned with the baseline under the recorded conflict observations for substantial portions of the search, and the decision traces for `swap_1791_5979` were identical over the traced interval from decisions 1191 through 2000 despite the eventual difference in total decision count. The experiments therefore support the existence of multiple pathways through which equivalent representations can produce different computational histories.

The central conclusion is consequently not that variable renaming inherently makes SAT solving harder or easier. Instead, the results show that a logically irrelevant change in representation can become computationally consequential when it interacts with the evolving state of a heuristic search algorithm. A deterministic solver can produce substantially different computational trajectories from semantically equivalent representations because the representation influences the concrete state from which subsequent decisions are made.

These findings support a broader view of CDCL solver behavior as path-dependent. The logical problem constrains the possible solutions, but it does not uniquely determine the sequence of internal states through which a particular deterministic implementation reaches one. Variable ordering, heuristic state, learned clauses, and other mutable structures can mediate this process, allowing a small representational perturbation to become a large difference in computational effort.

Within the scope of the five benchmark instances and the MiniSat 2.2.0 configuration examined here, variable renaming therefore provides a controlled demonstration of a broader computational principle: semantic invariance does not imply trajectory invariance. Two inputs can be logically interchangeable while remaining computationally distinct to the algorithm that processes them.

References

[1] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, “Chaff: Engineering an Efficient SAT Solver,” in *Proceedings of the 38th Design Automation Conference*, pp. 530–535, 2001. doi:10.1145/378239.379017.

[2] N. Eén and N. Sörensson, “An Extensible SAT-solver,” in *Theory and Applications of Satisfiability Testing (SAT 2003)*, Lecture Notes in Computer Science, vol. 2919, pp. 502–518, Springer, 2003. doi:10.1007/978-3-540-24605-3_37.

[3] A. Biere and M. J. H. Heule, “The Effect of Scrambling CNFs,” in *Proceedings of Pragmatics of SAT 2015 and 2018*, EPiC Series in Computing, vol. 59, pp. 111–126, 2019. doi:10.29007/9dj5.

[4] M. Vinyals, “Hard Examples for Common Variable Decision Heuristics,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 02, pp. 1652–1659, 2020. doi:10.1609/aaai.v34i02.5527.

[5] E. Dantsin and A. Wolpert, “An Isomorphism-Invariant Distance Function on Propositional Formulas in CNF,” *Journal on Satisfiability, Boolean Modeling and Computation*, vol. 14, no. 1, 2023. doi:10.3233/SAT-220006.